İZMİR DEMOKRASİ UNIVERSITY



MIS203 / WEEK 05

Functions And Modular Programs

Assoc. Prof. Dr. Kadir DEMİR

Res. Asst. Mehmethan ERDURAN

This Week In Python

1. Define and call functions
2. Use parameters and returned values
3. Check functions with small tests

The discount policy changes. Where should one edit the program?

What would the program need to know?

Functions Give A Task A Name

A function groups work that belongs together.

1. Define it once and call it where needed.
2. Parameters are the function’s inputs.
3. A return value is its result.

Where could a quote program reuse a calculation?

A Function for Subtotal

The caller receives the returned result.

```
def subtotal(quantity, unit_price):
    return quantity * unit_price

amount = subtotal(4, 125.0)
print(f"{amount:.2f} TRY")
```

Call it with 2 and 79.90.

print And return Have Different Jobs

A returned value can feed the next calculation.

1. print displays information to a person.
2. return sends a result to the caller.
3. Keep calculation and display separate.

Example: examples/02_return_value.py

Compose Small Functions

One function can use the result of another.

```
def subtotal(qty, price):
    return qty * price

def tax(amount):
    return amount * 0.20

amount = subtotal(3, 100)
print(amount + tax(amount))
```

What value does amount hold?

Local Names Stay Inside A Function

A function should have a clear boundary.

1. Pass required data as parameters.
2. Return the result that other code needs.
3. Avoid changing unrelated values outside the function.

Example: examples/04_scope.py

Check A Function With assert

A small check gives quick feedback.

```
def discount(amount, rate):
    return amount * rate

assert discount(100, 0.10) == 10
assert discount(0, 0.10) == 0
print("Checks passed")
```

Add a third check for a zero rate.

Why Does This print None?

A function without return sends back None.

```
def double(value):
    print(value * 2)

result = double(5)
print(result)
```

Rewrite the function to return 10.

Live Coding: Shipping Quote

Refactor one long calculation.

1. Write subtotal(), discount_rate() and shipping_fee().
2. Write a function that combines their results.
3. Add three boundary checks with assert.

Compare the structure with project_a_pricing_engine.py.

Project A: Pricing Engine

Separate input, rules and quote calculation.

projects/project_a_pricing_engine.py

Change one rule without rewriting the others.

Project B: Expense Analyzer

Use small functions for a running summary.

projects/project_b_expense_analyzer.py

Explain each function’s input and output.

In-Class Lab: Reusable Shipping Quote

Your Challenge

Assignment: week05/lab-quiz/lab_assignment.md

Submit in your repository: week05/lab-quiz/

Build it. Check a boundary. Explain one decision.

GitHub Checkpoint

Lab and quiz: week05/lab-quiz/

Keep weekly homework in week05/ outside that folder.

```
git add week05/lab-quiz/ README.md
git commit -m "week05: complete lab quiz"
git push
```

Show the lab, one test and the commit.

See You Next Week